ChatGPT

Write Website Code (1)

You

make the website GUI look a lot nicer

Worked for 1m 1s >

ChatGPT

Absolutely. I redesigned the GUI to make it feel much more like an actual educational website while **keeping the existing functionality and CISC requirements intact**.

Changes made

- Added clearer visual hierarchy with section headings.
- Added icons/emojis for navigation and actions.
- Added separators and better spacing.
- Redesigned the **Home** page as a simple landing page.
- Redesigned the **Converter** page with distinct input/result sections.
- Improved the **History** page.
- Made the **Learn** page easier to scan.
- Improved the **Practice** page with a clearer challenge area.
- Made correct/incorrect practice feedback more visually obvious.
- Kept the existing routes, dataclasses, inputs, history, tests, and conversion logic.
- Verified the modified file has **no syntax errors**.
- Verified there are **no lines over 119 characters**.

I've saved the redesigned version here:

One limitation: I could verify the Python syntax, but I can't render the actual Drafter webpage in this environment because the Drafter package isn't installed here. So the visual changes are based on the Drafter components already used by your project.

main_cisc_requirements_gui.py
Code

Take your curiosity further
Scan to get the ChatGPT app

You

add more color to the website so it doesn't look very bland

Worked for 31s >

ChatGPT

Done — I gave the website a much more colorful look without changing the converter, practice system, history, routes, or CISC requirements.

🎨 New look

- **Pico theme** for a more polished overall design.
- Soft **blue → purple → orange gradient** background.
- Purple/indigo headings.
- Colorful gradient buttons with hover effects.
- Rounded buttons and input fields.
- Purple focus highlights around inputs.
- Styled conversion-history area.
- Subtle fade transition between pages.
- Kept the design educational rather than making it overly flashy.

I also checked the file for syntax errors and confirmed the longest line is only **96 characters**, so it stays within your course's 120-character limit.

The custom styling uses Drafter's documented `set_website_style` and `add_website_css` features. drafter-edu.githu... +1

main_cisc_requirements_colorful.py
Code

You

main_cisc_requir...
Code

can you audit the code against the cisc requirements

Take your curiosity further
Scan to get the ChatGPT app

Worked for 10s >

ChatGPT

Yes. I audited the uploaded `main_cisc_requirements_technical.py` against the CISC requirements we've been using for your final project.

Overall result

The code meets almost all of the CISC requirements. I found **one clear requirement issue** that I would fix before submission: the test variables at the bottom of the file are global variables.

CISC Requirements Audit

Requirement	Status	Evidence / Notes
At least 7 routes	✅ Pass	There are **10 routes**: `index`, `home`, `converter`, `convert_number`, `history`, `clear_history`, `learn`, `practice`, `new_practice`, `check_answer`. ‹/› main_cisc_requirements_technical ‹/› main_cisc_requirements_technical
At least 5 pages	✅ Pass	Home, Converter, History, Learn, and Practice are all implemented as distinct pages/routes. ‹/› main_cisc_requirements_technical ‹/› main_cisc_requirements_technical ‹/› main_cisc_requirements_technical
State dataclass with ≥4 fields	✅ Pass	`State` has **6 fields**. ‹/› main_cisc_requirements_technical

Take your curiosity further
Scan to get the ChatGPT app

Requirement	Status	Evidence / Notes
Second dataclass	✅ Pass	`Conversion` is a second dataclass with 4 fields. ‹/› main_cisc_requirements_technical
State contains list/dict of second dataclass instances	✅ Pass	`conversion_history: list[Conversion]`. ‹/› main_cisc_requirements_technical
At least 4 State fields meaningfully modified	✅ Pass	`current_input`, `input_base`, `error_message`, `decimal_result`, `conversion_history`, and `practice_number` are modified by routes. ‹/› main_cisc_requirements_technical ‹/› main_cisc_requirements_technical
At least 3 input fields	✅ Pass	`TextBox("number_text")`, `SelectBox("input_base")`, and `TextBox("practice_answer")`. ‹/› main_cisc_requirements_technical ‹/› main_cisc_requirements_technical
At least 3 meaningful `if` statements	✅ Pass	There are many, including input validation, base selection, history display, and answer checking. ‹/› main_cisc_requirements_technical ‹/› main_cisc_requirements_technical ‹/› main_cisc_requirements_technical
At least 1 loop over second-dataclass instances	✅ Pass	`history_table` loops through the conversion records and accesses their fields. ‹/› main_cisc_requirements_technical

Take your curiosity further
Scan to get the ChatGPT app

Requirement	Status	Evidence / Notes
Legitimate purpose/functionality	✅ Pass	The website converts decimal/binary/hexadecimal values and teaches the conversion process. `</>` main_cisc_requirements_technical
No global variables	⚠️ **Needs fix**	See below.
Uses Drafter	✅ Pass	`from drafter import *`, routes, `Page`, `Button`, `TextBox`, etc. `</>` main_cisc_requirements_technical
Advanced techniques documented	⚠️ Mostly pass	The code uses several techniques beyond basic conversion, but whether they count as "advanced" depends on your instructor's definition.
Code should run/deploy	⚠️ Not fully verified	The file structure looks appropriate, but this audit does not prove that the actual Drafter server runs successfully.
Unit tests	✅ Pass	There are helper-function tests, edge cases, conversion tests, route tests, and practice tests. `</>` main_cisc_requirements_technical
Tests for routes where possible	✅ Pass	The file explicitly tests all 10 routes. `</>` main_cisc_requirements_technical
Types in function headers	✅ Pass	Functions consistently use type annotations. `</>` main_cisc_requirements_technical
Docstrings	✅ Pass	Functions have first-line docstrings.

Take your curiosity further
Scan to get the ChatGPT app

Requirement	Status	Evidence / Notes
Long lines <120 characters	✅ Pass	The displayed code does not appear to contain lines exceeding 120 characters.
`set_site_information`	✅ Pass	Author, description, sources, planning document, and links are provided. ‹/› main_cisc_requirements_technical
No obvious global test state object	✅ Pass	Unlike the earlier version, there is no `test_state = State(...)`.
Descriptive commits	❓ Cannot verify	This requires checking your Git repository.
Planning document in repo	⚠️ Needs repo verification	The code references `"Copy of CS1 Final Project 1_ Design.pdf"`. ‹/› main_cisc_requirements_technical
Video included	❓ Cannot verify	Requires checking the repository.
AI conversations uploaded	❓ Cannot verify	Requires checking the repository.

⚠️ Main issue: global test variables

This is the one thing I would definitely change.

At the bottom, these assignments create module-level/global variables:

```
hex_result = convert_number(...)
invalid_result = convert_number(...)
wrong_result = check_answer(...)
new_question_state = State(...)
```

For example, `hex_result` is created at the module level. ‹/› main_cisc_requirements_technical

The rubric requirement says:



Take your curiosity further

Scan to get the ChatGPT app



> **No global variables (global constants okay).**

So even though these variables are only being used for tests, they technically violate that requirement.

How serious is it?

Minor, but worth fixing.

The earlier version of your project had a `test_state` global, and we removed that. This new version accidentally introduces several smaller global variables through the tests.

The simplest approach would be to restructure those tests so they don't leave named variables at global scope.

One thing I'd double-check with your instructor

The loop requirement

You currently have:

```
for index in range(len(conversions)):
    conversion = conversions[index]
```

This is absolutely a loop over your `Conversion` instances, and it accesses their attributes.

`</>` main_cisc_requirements_technical

So **I would count this as satisfying the requirement**.

However, if your instructor specifically interprets the requirement as:

> "Loop directly over the `State` attribute containing the second dataclass instances"

then they might prefer seeing the loop directly operate on `state.conversion_history`.

I don't think that's necessarily required based on the wording we've been working from, but it's the one rubric interpretation I'd keep in mind.

Take your curiosity further
Scan to get the ChatGPT app

Final grade-style assessment

🟢 Requirements clearly satisfied

- Routes: **10/7**
- Pages: **5+**
- State fields: **6**
- Second dataclass: **yes**
- List of second dataclass instances: **yes**
- 4+ State fields modified: **yes**
- 3+ input fields: **yes**
- 3+ meaningful conditionals: **yes**
- Loop: **yes**
- Legitimate functionality: **yes**
- Drafter: **yes**
- Unit tests: **yes**
- Type annotations: **yes**
- Docstrings: **yes**
- Site information: **yes**

🟡 Needs attention

1. **Global test variables** — fix these before submitting.
2. **Actual runtime/deployment** — run the website and verify it works.
3. **Planning document filename** — make sure the referenced PDF actually has that exact name in your repository.
4. **Advanced-technique documentation** — make sure your planning/documentation explains any techniques your instructor considers beyond the course material.
5. **Git commits, video, and AI-conversation files** — these cannot be verified from this Python file alone.

🔴 Nothing else obvious is failing

My recommendation: fix the global test-variable issue, then do a final runtime test. After that, the Python file itself looks very close to submission-ready from the CISC requirements perspective.

Take your curiosity further
Scan to get the ChatGPT app

You

can you make more descriptive comments with argments and return types, and fix changes like global variables

ChatGPT

Absolutely. I made the requested changes to the uploaded code.

Changes made

- **Expanded function docstrings** to include:
  - `Args:` descriptions for parameters
  - `Returns:` descriptions for return values
  - Clear explanations of what each function does
- **Fixed the global-variable issue** in the tests.
  - The previous `hex_result`, `invalid_result`, and `wrong_result` variables were removed.
  - The `new_question_state` test state is now local to a test function.
  - The only remaining module-level variables are `SANS` and `MONO`, which are global constants used for styling.
- Added a comment explaining why test state is kept local.
- Preserved the existing functionality and CISC requirements.
- **Python syntax check passed.**
- I also checked module-level assignments: only the two styling constants remain.

Updated file

This version should be a stronger submission for the requirements concerning **documentation, arguments/return types, and no global variables**.

main_cisc_requirements_technical_fixed.py
Code

Take your curiosity further
Scan to get the ChatGPT app

You

export this conversation in human readable format such as .txt or .md file

Analysis paused

ChatGPT

Done. I exported the conversation in two human-readable formats:

The Markdown version is formatted with headings and sections for easier reading.

cisc_conversation_export.txt
Document

cisc_conversation_export.md
Document

You

add the chats from the continued conversation history

Worked for 5s

Analysis errored

Analyzed

ChatGPT

Done. I added the continued project conversation history covering the CISC fixes, audits, GUI improvements, color/styling changes, documentation improvements, and global-variable fixes.

Updated exports:

cisc_conversation_export_updated.md
Document

cisc_conversation_export_updated.txt
Document

Take your curiosity further
Scan to get the ChatGPT app